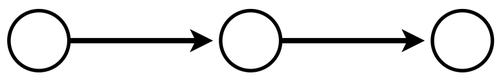
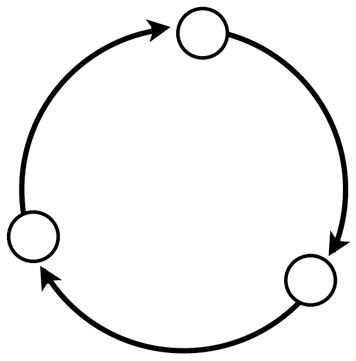
29. Recursion

The Concept of Recursion

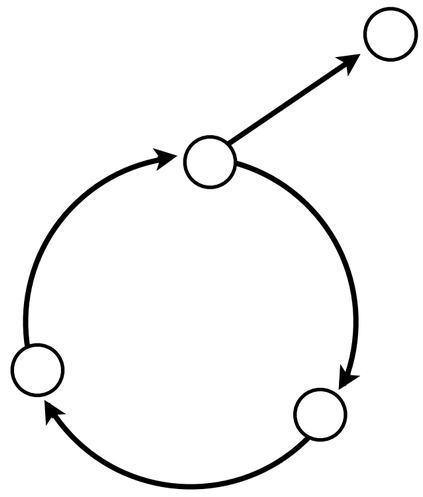
A recursive algorithm repeats some process with slight modifications until some goal is reached. "Recursive" means repeating. Recursive algorithms occur not only in computation, but also in natural processes.



A finite sequence of states.



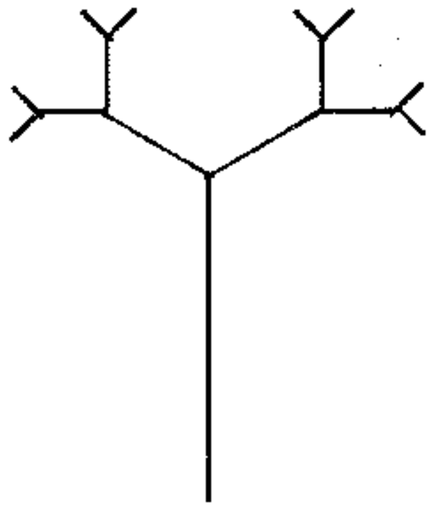
A loop of states which repeat infinitely.



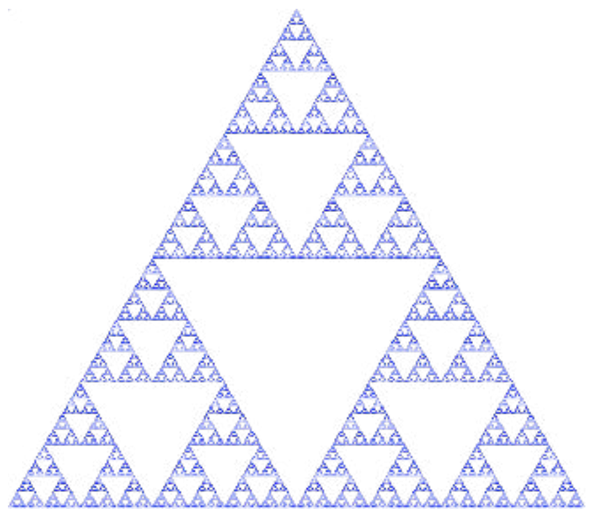
A loop which repeats a finite number of times until an exit condition is reached.

Recursive algorithms can be used to generate drawings with elements that repeat, but also change a little bit, after each loop.

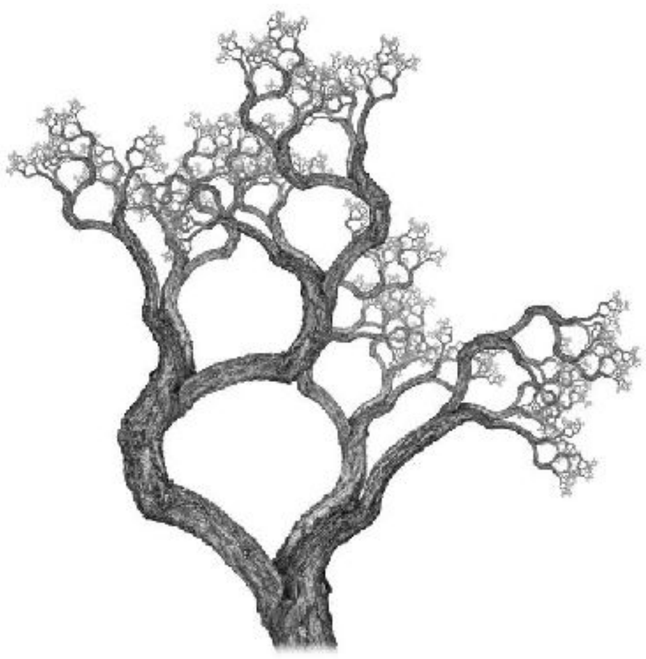
```
TO TREE :SIZE
FORWARD :SIZE
IF :SIZE <1 STOP ELSE TWO-
TREES SIZE/2
BACK :SIZE
END
```



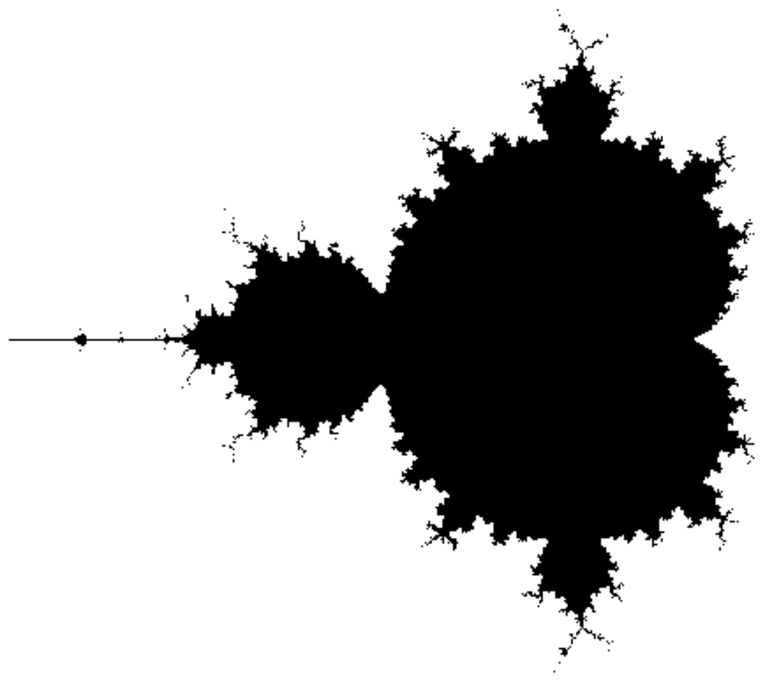
This program written in LOGO describes an algorithm for drawing simple trees.



How would you generate Sierpinksi's triangle?



The same technique can be used to draw more realistic-looking trees…

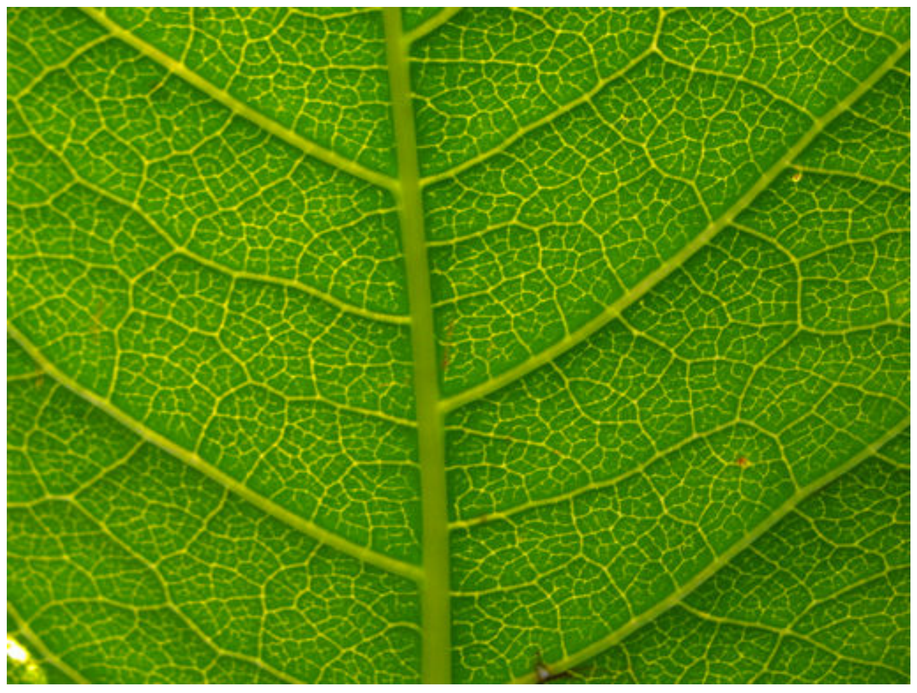


… or bizarre and complex fractals.

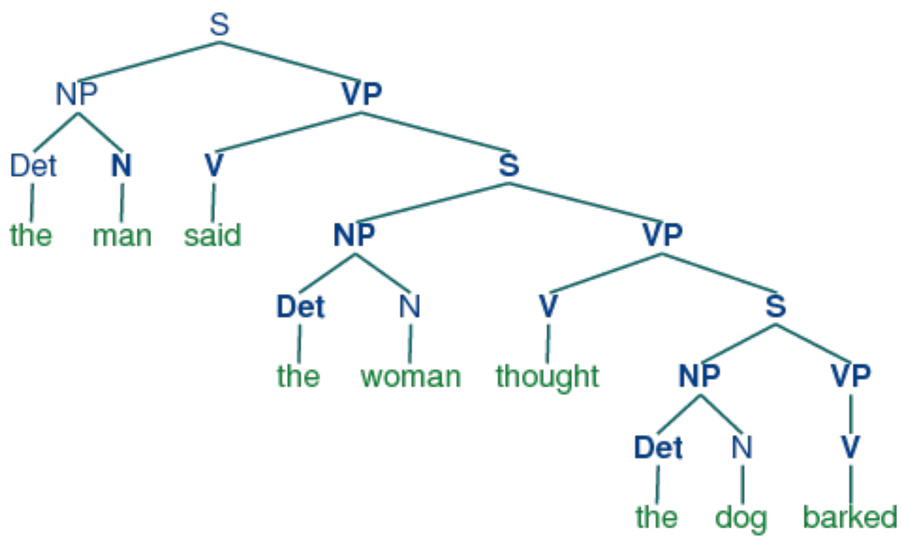
Recursion in Nature

The natural world is filled with recursive structures. Consider a tree: how are the branches generated? How is each leaf on a branch generated? How is each cell is a leaf generated?









Cell growth works by repeating the same kind of operation over and over, with slight modifications made for a single kind of structure (an organ, a leaf), and larger modifications.

Famously, language is also thought to have a recursive structure. Sentences are formed by recursively combining parts of speech.

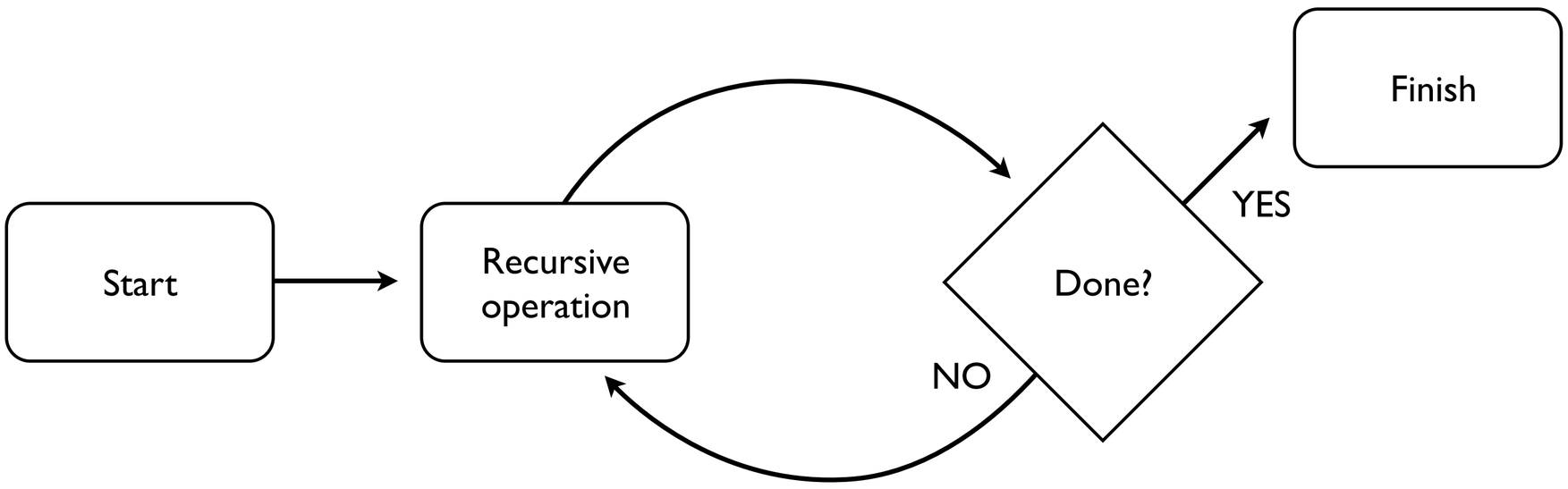
Both cellular growth and sentence structures are driven by recursive operations. But do they have the same root cause?

Perhaps language is recursive *because* biological growth is recursive. Somehow the biological structure of the brain requires it. Perhaps. A more plausible alternative is that both recursive for the same reason. Recursive algorithms are the central mechanism for generating indefinite complexity from a finite base of instructions.

The same pressures which made biology recursive are also what make language recursive, and it is the same pressure which makes computer programs recursive. All face the same kinds of ecological pressure: infinite ends, finite means.

The Architecture of Recursion

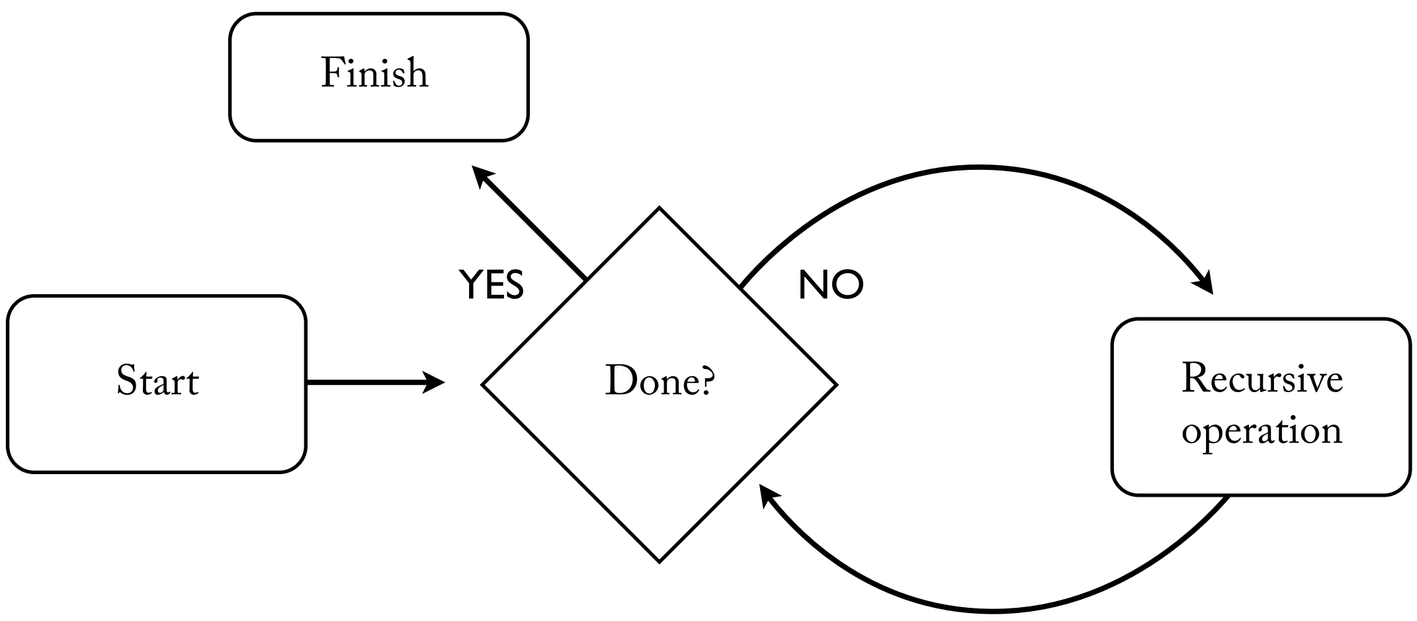
Recursive algorithms have a characteristic architecture. Here is a typical design:



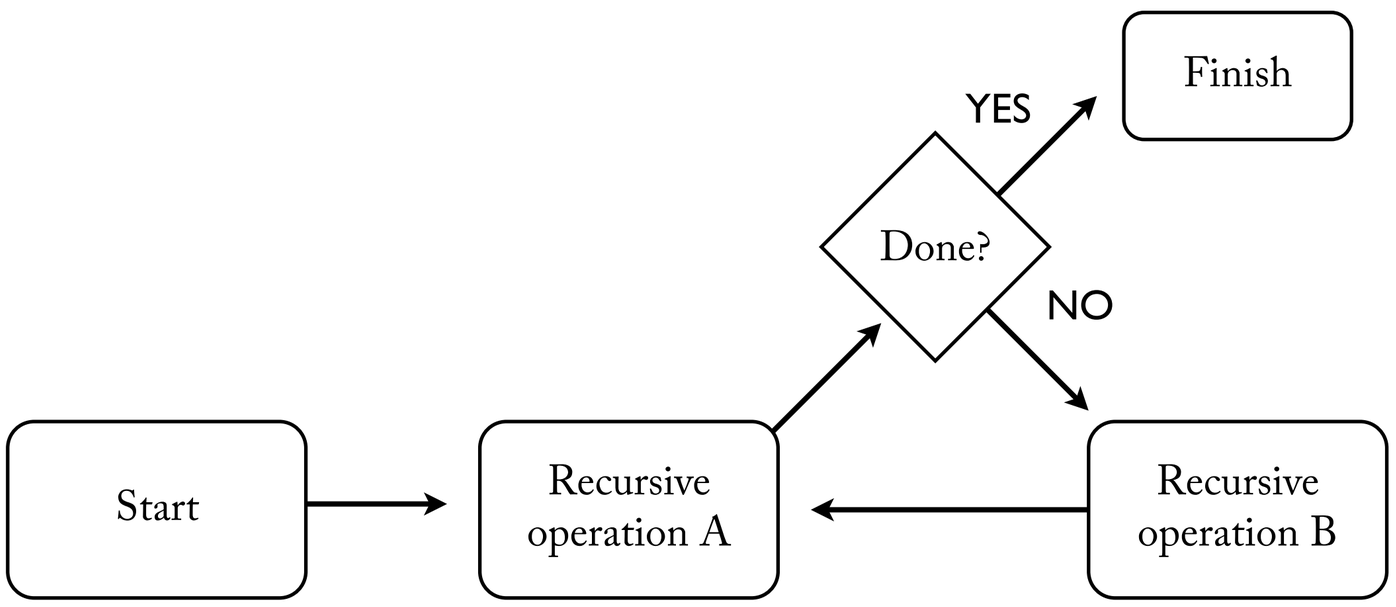
A recursive algorithm has three key elements:

(1) a **recursive operation** which is a repeated;
(2) a way to **test** whether the process should be repeated again;
(3) an **exit operation** when the process is complete.

The three elements may appear in nearly any order. You'll have to find the order best suited for the problem.



Often the recursive operation will itself involve several discrete operations, and these can be re-arranged as well. The underlying conceptual structure is the same.



Note: these diagrams are **flow charts**, a common system for describing algorithms. See the Section "Flow Charts" for more about creating and reading these diagrams.

Case Study: 2*x* T

The goal is to design a TM computes $2x$ in Tally. For present purposes assume $x \geq 1$. Here it is instructive to remember what multiplication really means. Multiplying x by 2 is equivalent to adding 2 to 0, x times. For example:

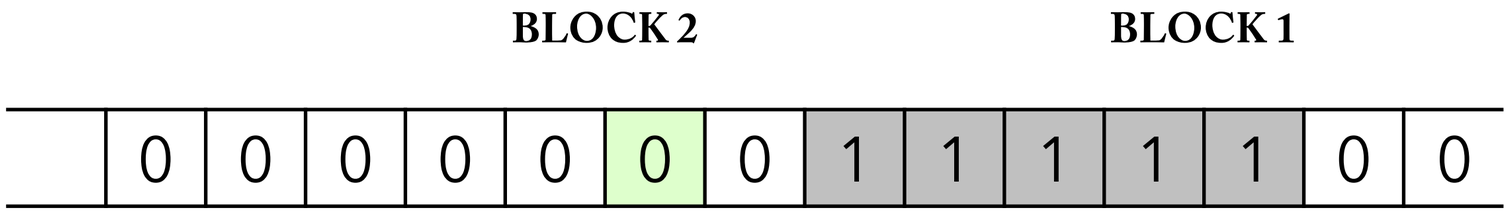
$$\begin{aligned} 2 \times 3 &= 0 + 2 \\ &= 2 + 2 \\ &= 4 + 2 \\ &= 6 \end{aligned}$$

3 times

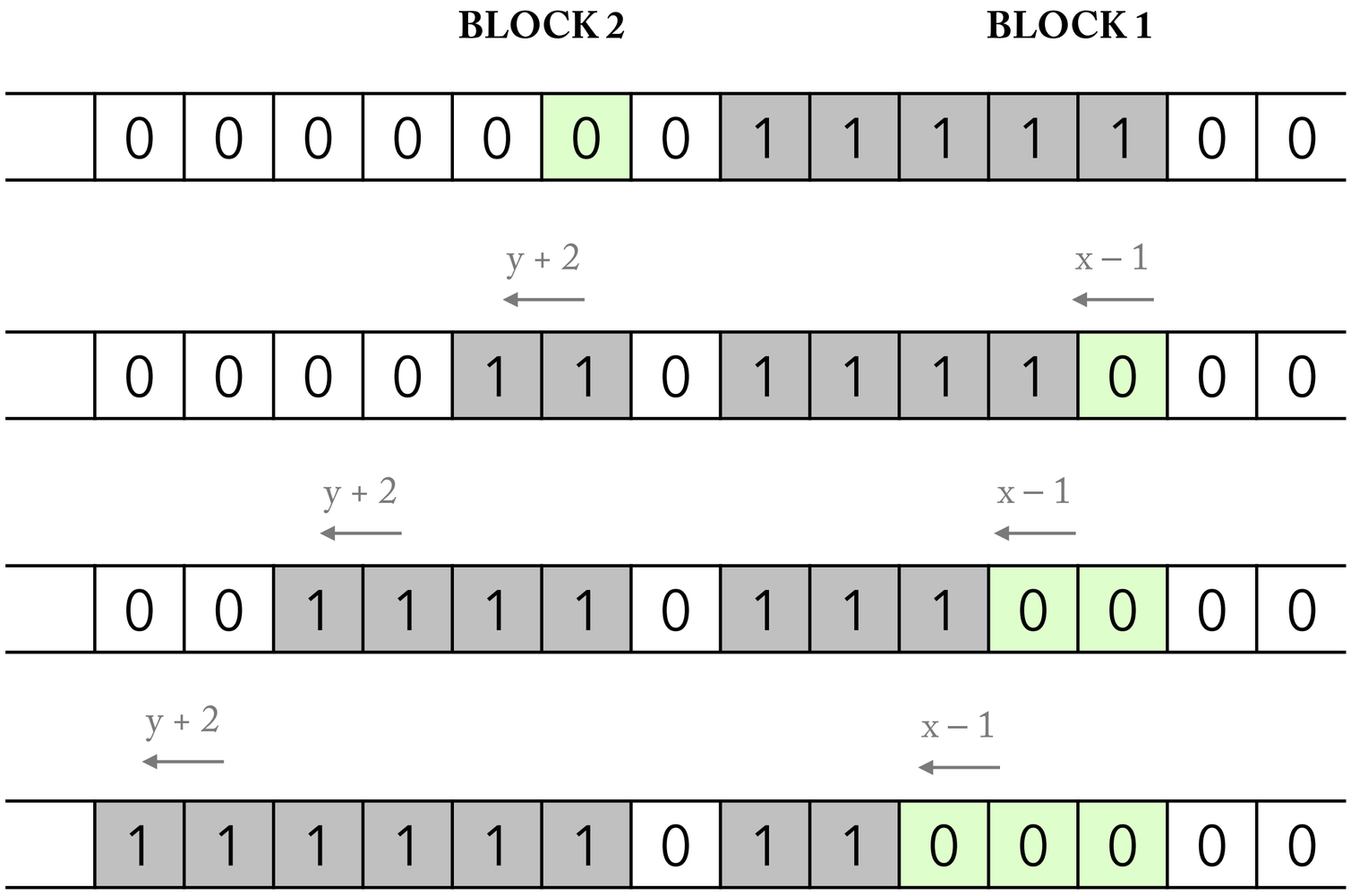
$$\begin{aligned} 2 \times 5 &= 0 + 2 \\ &= 2 + 2 \\ &= 4 + 2 \\ &= 6 + 2 \\ &= 8 + 2 \\ &= 10 \end{aligned}$$

5 times

To approach this problem, let us imagine two blocks on the tape. Block 1 (B1) represents the argument x. Block 2 (B2) represents the value $2x$. B1 starts with a value, but B2 starts empty.

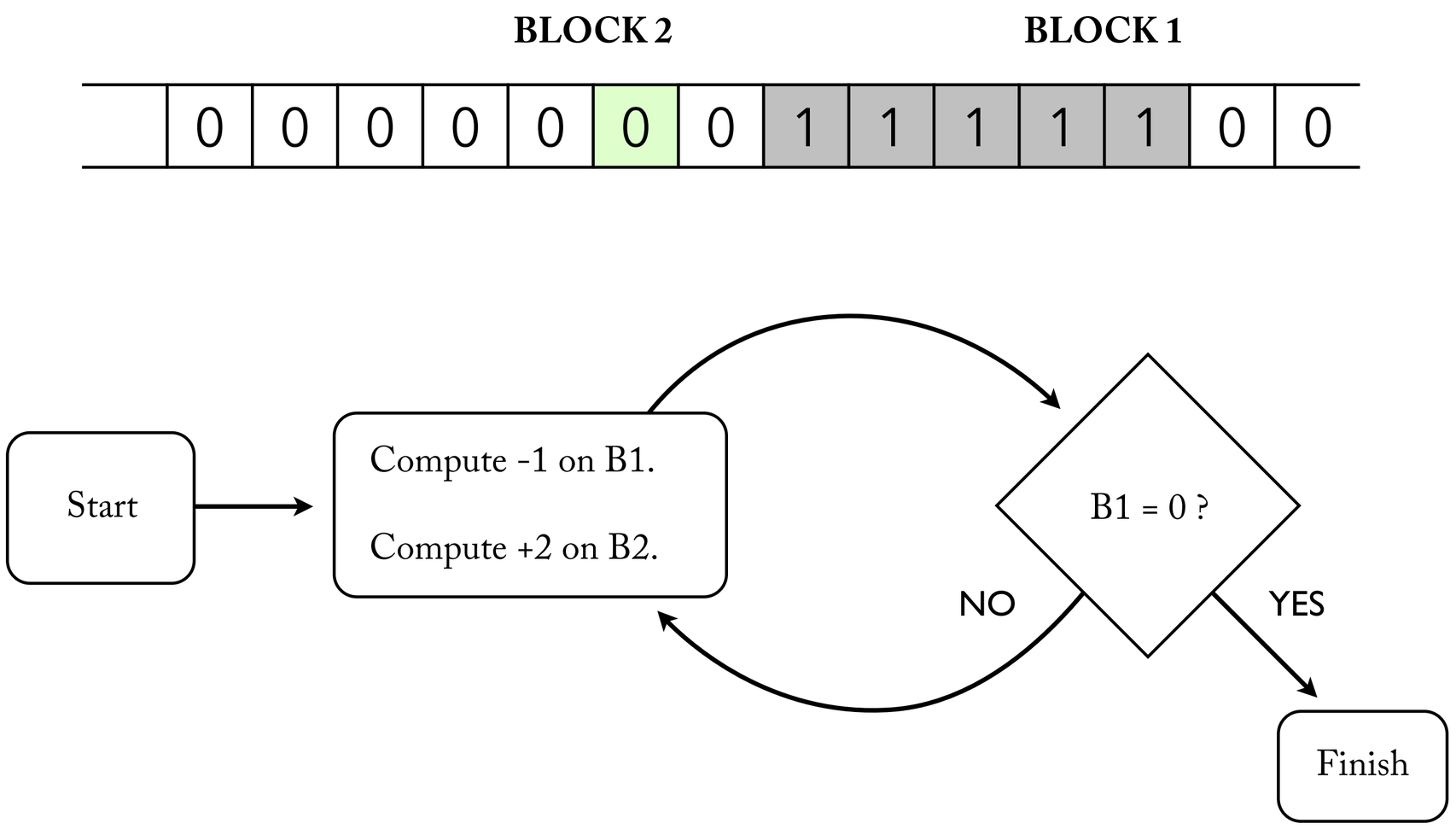


The strategy is this: let us repeatedly remove 1 from B1, and each time we do so, add 2 to B2. In other words, each time we compute x-1 on B1, we'll then compute y+2 on B2. When B1 is empty, B2 should have twice as many cells filled as B1 did originally.

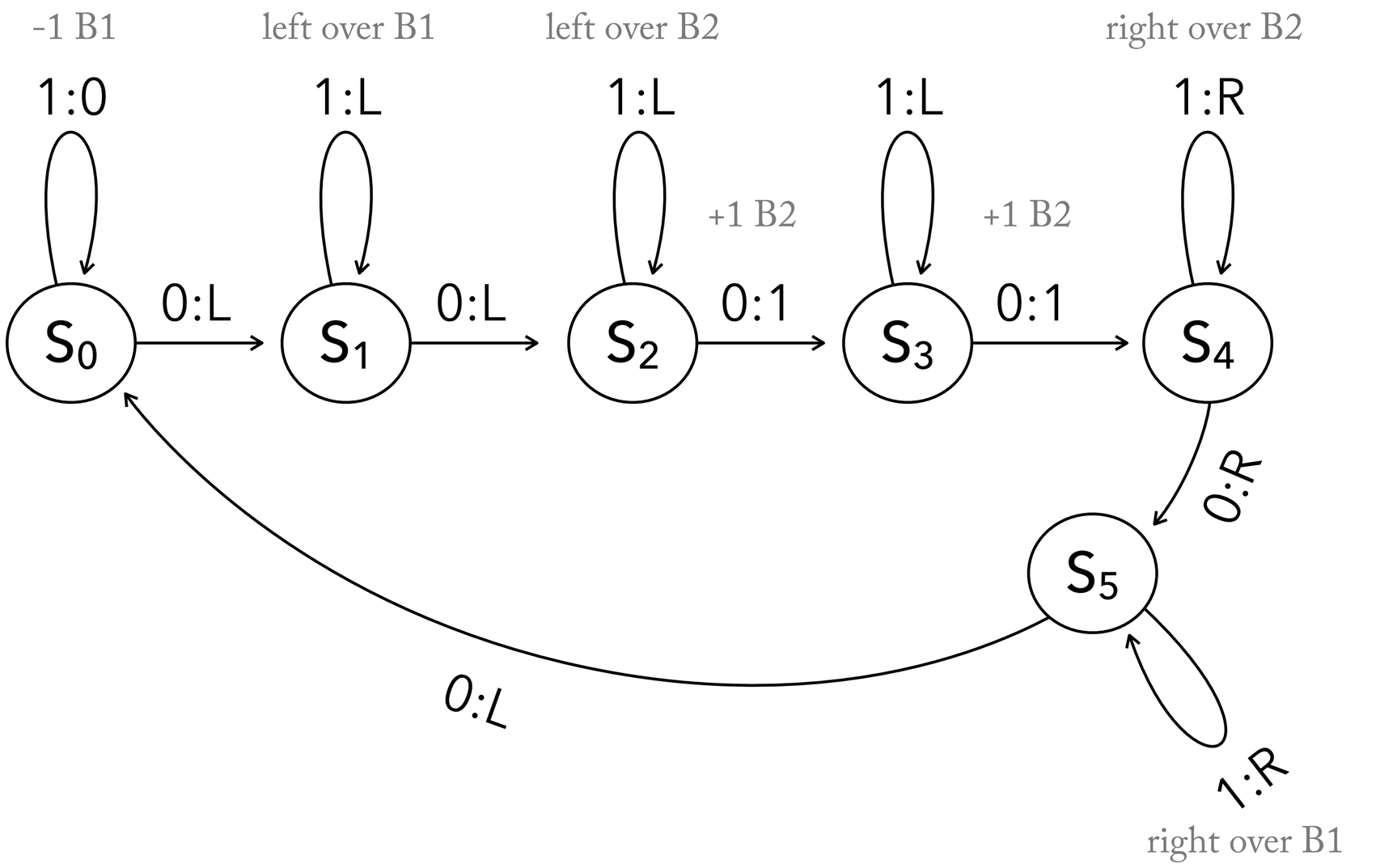


Here B1 functions as **counter**: it tells us how far we are into the comptuation. B2 functions as the **answer.** Nearly all recursive algorithms will include both elements. **Note:** here I remove 1 from the right side of my counter, but the left side would work just as well.

Before drawing up our TM, let us convert our strategy into an explicit flow chart:

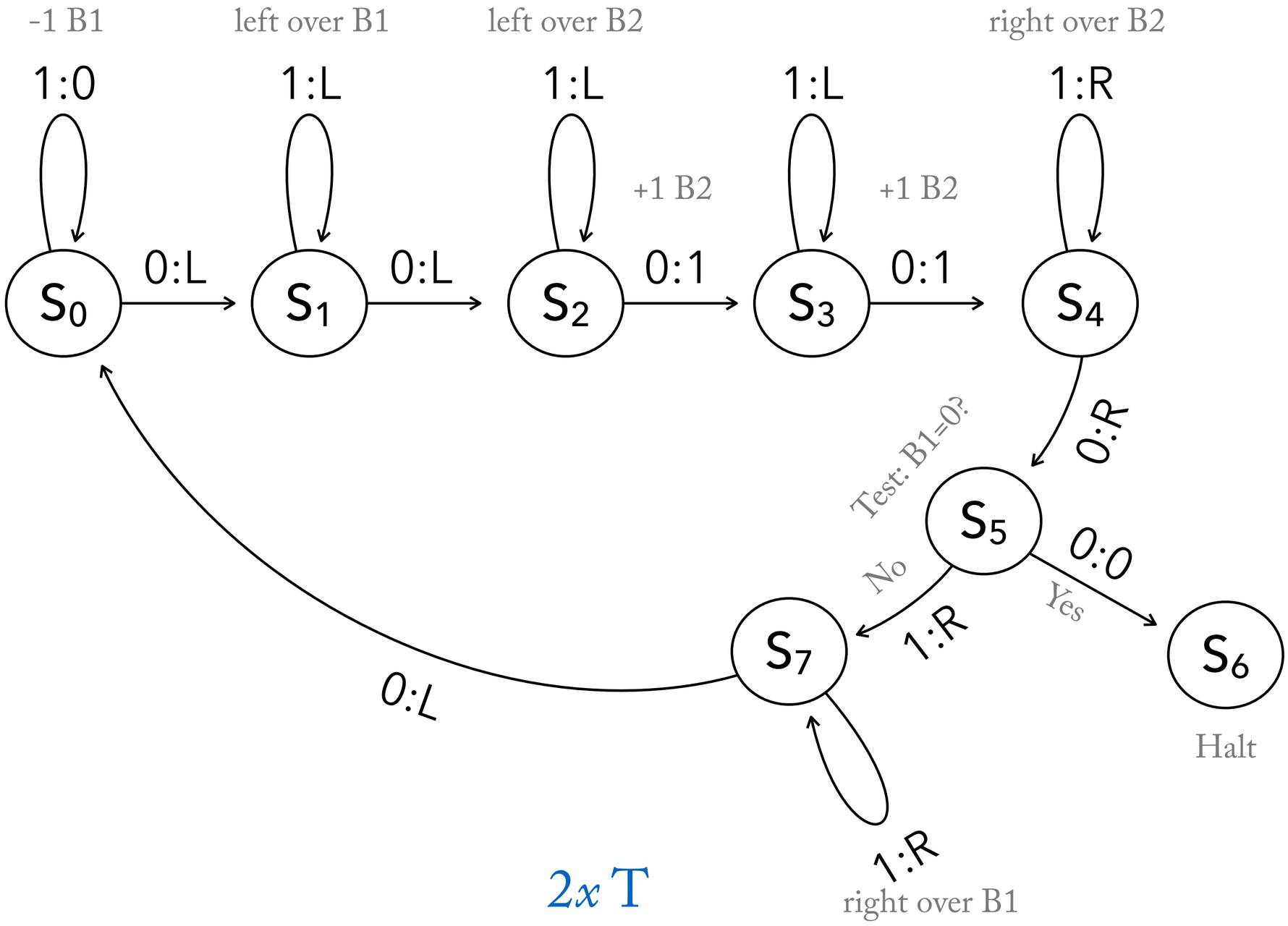


To turn this design into a TM, a good starting place is just to put the critical elements into a loop: (i) -1 from B1, (ii) +1 to B2, (ii) +1 to B2.



The problem with this design is that it contains no **test** or **exit operation**. As a result, it loops indefinitely. Can you predict what will happen to this machine after B1 is empty?

Finally, here is a fully annotated *2x* Tally TM. It's just like the machine above, but builds in a test to see when B1 is empty, and an explicit exit condition if it is.



Thinking through *x*+*y* T

How can we break addition up into a simple, repeating operation? Remember that addition is just successor applied over and over

```
2 + 3  = 2 + 1
       = 3 + 1
       = 4 + 1
       = 5
```

3 times

```
4 + 5  = 4 + 1
       = 5 + 1
       = 6 + 1
       = 7 + 1
       = 8 + 1
       = 9
```

5 times

The next step is to work through a strategy. Here's a start…

